



Universidad
Indoamérica

UNIDAD 6 · FUNDAMENTOS DE SISTEMAS OPERATIVOS

Señales y memoria compartida

Universidad Tecnológica Indoamérica

Laboratorio TD5

Señales: avisos asíncronos a un proceso

El núcleo

SIGSEGV (acceso inválido), SIGBUS, SIGPIPE (tubería sin lector), SIGCHLD (terminó un hijo), SIGALRM

La terminal

Ctrl-C → SIGINT, Ctrl-\ → SIGQUIT, Ctrl-Z → SIGTSTP, cerrar → SIGHUP

Otro proceso

`kill(pid, sig)` y el comando `kill`: SIGTERM, SIGKILL, SIGUSR1...

Cambian el estado del proceso: lo terminan, lo detienen, lo reanudan... o ejecutan una función suya.

Envío asíncrono, señales pendientes y el momento de la captura

- El envío puede ocurrir en cualquier momento, esté o no ejecutando el destino
- **Pendiente**: recibida pero no entregada todavía (bloqueada, o el núcleo aún no la revisó)
- Se entrega al **volver a modo usuario**: tras una llamada al sistema o un cambio de contexto

Precisión al original ("no existe cola de señales"): es cierto para las señales *estándar* —una pendiente es un bit, las repetidas se pierden—, pero las de **tiempo real** (`SIGRTMIN` ... `SIGRTMAX`) sí se encolan y pueden llevar un dato (`sigqueue`).

kill y las señales más comunes

C

```
int kill(pid_t pid, int sig);  
/* pid > 0: that process · 0: my process group  
   < -1: group -pid · -1: every process I may signal  
   errors: EINVAL, EPERM, ESRCH */
```

Corrección al original (diapositiva 97):
decía que SIGHUP, SIGINT, SIGTSTP... son "no enmascarables y no bloqueables". Solo **SIGKILL** y **SIGSTOP** no se pueden capturar, ignorar ni bloquear; todas las demás, sí.

SEÑAL	POR OMISIÓN
SIGHUP · SIGINT · SIGTERM	terminar
SIGQUIT · SIGSEGV · SIGBUS	terminar + core
SIGKILL	terminar (siempre)
SIGPIPE · SIGALRM · SIGUSR1 · SIGUSR2	terminar
SIGSTOP · SIGTSTP	detener
SIGCONT	continuar
SIGCHLD	ignorar

Pendientes, bloqueadas y entregadas

señal	n	pendiente	bloqueada	acción	por omisión
SIGINT (Ctrl-C)	2	0	—	SIG_DFL	terminar
SIGQUIT (Ctrl-\)	3	0	—	SIG_DFL	terminar + core
SIGKILL	9	0	imposible	SIG_DFL	terminar
SIGUSR1	10	1	sí	manejador	terminar
SIGSEGV	11	0	—	SIG_DFL	terminar + core
SIGPIPE	13	0	—	SIG_DFL	terminar
SIGALRM	14	0	—	SIG_DFL	terminar
SIGCHLD	17	0	—	SIG_DFL	ignorar
SIGCONT	18	0	—	SIG_DFL	continuar
SIGSTOP	19	0	imposible	SIG_DFL	detener
SIGTSTP (Ctrl-Z)	20	0	—	SIG_DFL	detener
SIGRTMIN	34	0	—	SIG_DFL	terminar

proceso

dormido en pause / sigsuspend

lo que pasó

- sigprocmask(SIG_BLOCK, {SIGUSR1}); while (!flag)...
- kill(pid, SIGUSR1): el núcleo marca SIGUSR1 como pendiente
- llega en el peor momento, pero queda PENDIENTE (bloqueada)
- sigsuspend(máscara sin SIGUSR1): desbloquea y espera en UNA sola operación

Manejadores: de `signal` a `sigaction`

C

```

sighandler_t signal(int sig, sighandler_t f); /* obsolete; SIG_ERR on error */
int sigaction(int sig, const struct sigaction *act,
              struct sigaction *old);
struct sigaction {
    void (*sa_handler)(int); /* or SIG_DFL, SIG_IGN */
    void (*sa_sigaction)(int, siginfo_t *, void *);
    sigset_t sa_mask; /* blocked during the handler */
    int sa_flags; /* SA_RESTART SA_SIGINFO SA_NOCLDSTOP... */
};

```

- Durante el manejador, la misma señal queda bloqueada
- Al volver, el programa sigue donde estaba. `sa_flags`: `SA_RESTART` reanuda la llamada interrumpida (si no, `EINTR`, unidad 2); `SA_SIGINFO` usa `sa_sigaction`; `SA_NOCLDSTOP`: `SIGCHLD` solo cuando un hijo termina, no cuando se detiene o continúa
- Dentro solo funciones **async-signal-safe** (`write` sí, `printf` y `malloc` no) y variables `volatile sig_atomic_t`
- `signal()` tiene semánticas distintas según el sistema: usa `sigaction`

Esperar y programar: `pause`, `alarm`, `sleep`

- `pause()`: duerme hasta que una señal lo termine o ejecute un manejador; siempre devuelve `-1`
- `alarm(s)`: SIGALRM dentro de `s` segundos (por omisión, termina)
- `sleep(s)`: suspende `s` segundos

Actualización (diapositiva 101): el original decía que `sleep` se implementa con `pause` + `alarm` y que no se deben mezclar. En glibc/Linux, `sleep` usa `clock_nanosleep` y no toca la alarma. Hoy se prefieren `nanosleep` y los temporizadores `timer_create` o `timerfd`.

Ejemplos del original... y su error escondido

c

```
/* slide 102: mysleep */
signal(SIGALRM, do_nothing);
alarm(s);
pause();          /* what if SIGALRM arrives BEFORE pause? */

/* slide 103: ping-pong, parent and child after fork() */
kill(getppid(), SIGUSR1);
pause();          /* same race: the answer may come first */
```

- Si la señal llega **entre** la preparación y `pause()`, el manejador corre antes y `pause` espera para siempre (diapositiva 105)
- Error difícil de detectar y reproducir: depende del planificador (como la carrera de `mypipe`, unidad 5)
- Conclusión del original (diapositiva 104): **no uses** `signal()` / `pause()` para sincronizar

Bloquear, comprobar y sigsuspend

C tds/td5/mysignal.c

```
/* Block SIGUSR1 outside sigsuspend: a signal arriving
   and the wait stays pending instead of being lost (th
sigemptyset(&block);
sigaddset(&block, SIGUSR1);
exit_if(sigprocmask(SIG_BLOCK, &block, &wait_mask) == -1);
sigdelset(&wait_mask, SIGUSR1);

while (received < max)
    sigsuspend(&wait_mask);          /* atomically: unbl
```

sigprocmask bloquea; sigsuspend **desbloquea y duerme en una sola operación atómica.**

Operaciones con conjuntos: sigemptyset, sigfillset, sigaddset, sigdelset, sigismember.

Salida real

```
$ ./mysignal 2 & P=$!; sleep 0.3; kill -USR1 $P; sleep
0.2; kill -STOP $P; kill -USR1 $P; sleep 0.2; kill -CONT
$P; sleep 0.2; kill -USR1 $P; wait $P; echo "exit status:
$? (128 + 10 = killed by SIGUSR1)"
pid 357338: waiting for SIGUSR1 (2 times)
handler: received signal 10
handler: received signal 10
default action restored: the next SIGUSR1 terminates me
bash: line 1: 357338 User defined signal 1
./mysignal 2
exit status: 138 (128 + 10 = killed by SIGUSR1)
```

Señal + memoria: el emisor y el receptor

Salida real

```
$ ./receiver & sleep 0.3; echo 'hello from the sender' | ./sender; wait
receiver 357391: waiting for SIGUSR1
sender: the receiver is 357391
received: hello from the sender
```

- El receptor escribe su PID al principio de un archivo mapeado y espera SIGUSR1 con `sigsuspend`
- El emisor lee el PID, escribe el mensaje en el mismo archivo mapeado y envía la señal: la señal **avisa**, la memoria **transporta**

Hoy además: `signalfd` (leer señales como datos de un descriptor), `pidfd_send_signal` (señalar sin carreras de PID reutilizados).

Compartir memoria entre procesos

- La memoria virtual aísla: cada proceso tiene sus páginas (unidad 4)
- Compartir = hacer que páginas virtuales de **dos procesos** apunten al **mismo marco físico**
- Hace falta un **punto de encuentro**: un archivo o un nombre

mmap +
archivo

el archivo es el punto de encuentro y guarda los datos (persiste)

shm_open +
mmap

POSIX, un nombre en /dev/shm (en RAM), recomendado hoy

ftok +
shmget +
shmat

System V, clave numérica; el contenido del archivo no se usa

Proyectar un archivo en memoria: `mmap`

C

```
void *mmap(void *addr, size_t len, int prot, int flags,  
           int fd, off_t offset); /* MAP_FAILED on error */  
int munmap(void *addr, size_t len);  
int msync(void *addr, size_t len, int flags);
```

- `prot`: `PROT_NONE`, `PROT_READ`, `PROT_WRITE`, `PROT_EXEC`
- `flags`: `MAP_SHARED`, `MAP_PRIVATE` (copia en escritura), `MAP_ANONYMOUS`
- Errores: `EACCES` (archivo no regular, o el modo del descriptor no permite lo pedido: `O_WRONLY` y `O_APPEND` no sirven), `ENOMEM`

Corrección al original (diapositiva 118):

`len` no necesita ser múltiplo del tamaño de página (se redondea hacia arriba); el que sí debe serlo es `offset`. Y acceder más allá del final del archivo no da `SIGSEGV` sino **SIGBUS**: lo muestra la siguiente animación con la salida real.

Compartir memoria mediante un archivo

C tds/td5/sender.c

```
int fd = open(FILENAME, O_RDWR);
exit_if(fd == -1, "open " FILENAME);
char *shared = mmap(NULL, SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
exit_if(shared == MAP_FAILED, "mmap");
exit_if(close(fd) == -1, "close");

...

shared[off] = '\\0';

exit_if(msync(shared, SIZE, MS_SYNC) == -1, "msync"); /* optional: also push to the file */
exit_if(kill(pid, SIGUSR1) == -1, "kill");
exit_if(munmap(shared, SIZE) == -1, "munmap");
```

- `O_RDWR` : `PROT_WRITE` + `MAP_SHARED` exige un descriptor de lectura y escritura
- Tras el `mmap` se puede cerrar el descriptor: la proyección sigue viva
- Escribir en `shared[i]` es escribir en el archivo: el receptor, que proyecta el mismo archivo, lo ve
- `munmap` deshace la proyección; `msync` fuerza la copia al disco

Los límites de `mmap` : `SIGBUS` y `PROT_*`

página 0 (0 ... 4095)

zona legible/escribible pero fuera del archivo: se ve 0, no se guarda

archivo: 128 B

bytes escritos con `map[i] = 'x'`

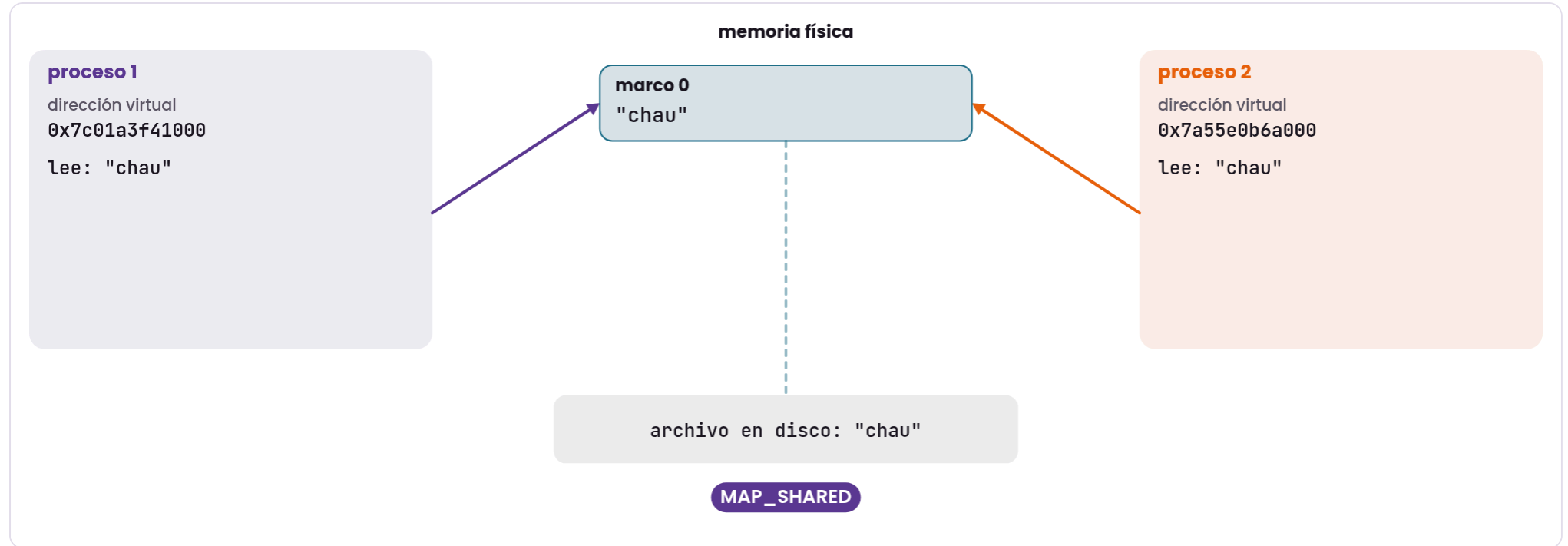
página 1 (4096 ... 8191)

sin archivo detrás → `SIGBUS`

`SIGBUS @ 4096`

tamaño del archivo después: 128 B

Dos procesos, un marco físico



Memoria compartida System V: `ftok` → `shmget` → `shmat`

C tds/td5/shm_counter.c

```
key_t key = ftok(KEYFILE, KEYID);
exit_if(key == -1, "ftok");
int id = shmget(key, SIZE, IPC_CREAT | 0644);
exit_if(id == -1, "shmget");

if (argc > 1 && strcmp(argv[1], "--remove") == 0) {
    exit_if(shmctl(id, IPC_RMID, NULL) == -1, "shmctl I");
    printf("segment %d removed\n", id);
    return EXIT_SUCCESS;
}

int *counter = shmat(id, NULL, 0);
exit_if(counter == (void *)-1, "shmat");
counter[0]++; /* a fresh
printf("segment %d (key 0x%08x): counter = %d\n", id, (
exit_if(shmdt(counter) == -1, "shmdt");
```

Salida real

```
$ ./shm_counter; ./shm_counter; ./shm_counter;
./shm_counter --remove
segment 65593 (key 0x4605476d): counter = 1
segment 65593 (key 0x4605476d): counter = 2
segment 65593 (key 0x4605476d): counter = 3
segment 65593 removed
```

- El segmento sobrevive a los procesos hasta `shmctl(IPC_RMID)`; `ipcs -m` los lista
- Cada proceso lo ve en otra dirección: **nunca guardes punteros** en él
- `IPC_PRIVATE`: segmento sin clave · `SHM_RDONLY`: solo lectura · `shmctl`: `IPC_STAT` lee, `IPC_SET` cambia permisos, `IPC_RMID` destruye

Compartir memoria exige sincronizar

Sin operación atómica

```
( slot = seg[0] + 1; seg[0] = slot; ):
```

Salida real

```
$ for i in 1 2 3 4 5; do ./shm_race 5 | cut -d: -f1; done
19 processes registered (expected 32)
28 processes registered (expected 32)
25 processes registered (expected 32)
23 processes registered (expected 32)
27 processes registered (expected 32)
```

Con `__atomic_add_fetch`:

Salida real

```
$ ./shm_fork
16 processes registered (expected 16): 357564 357566
357567 357572 357568 357571 357569 357565 357574 357573
357570 357576 357578 357575 357577 357579
```

Es el tema de la unidad 7: condiciones de carrera, mutex, semáforos.

TD5 resuelto + desafío con IA

- `mysignal` con `sigsuspend`; el emisor y el receptor
- `mmap_limits` y las cinco protecciones
- `shm_counter`, `shm_fork` y la carrera de `shm_race`

Bash

```
make -C tds && tds/tests/run.sh td5
```



Universidad
Indoamérica

PARA LLEVAR

Tres preguntas

1. ¿Por qué tres SIGUSR1 bloqueadas ejecutan el manejador una sola vez?
2. ¿Qué problema resuelve `sigsuspend` que `pause` no?
3. ¿Por qué no se guardan punteros en memoria compartida?